

Individual Final Project Report

Course: Deep Learning

Student: Mohammed Ismail Sarfaraz Shaik

Project: Automatic Image Restoration and Colorization using Deep Convolutional Autoencoders

1. Introduction

This project aims to solve the problem of automatic image colorization, a task that involves predicting plausible color channels for a given grayscale input without manual intervention. Traditional colorization requires human artists to manually assign colors to specific regions. In contrast, our automated approach leverages deep learning to learn the statistical mapping between grayscale intensity and color distribution from a large dataset.

Unlike classification tasks where a single label is correct, colorization is an ill-posed problem because a single grayscale object (e.g., a shirt) could validly be many different colors. To address this, our group developed a deep learning pipeline. My specific focus was the **Colorization Sub-system**, where I engineered an end-to-end deep learning pipeline using Autoencoders. This report details my work in developing and comparing two distinct U-Net architectures and deploying a final solution.

2. Description of Individual Work

My individual contribution focused on the generative colorization task. The core challenge was to map a 1-channel Grayscale image L to a 2-channel Color map (a, b) in the CIE Lab color space.

2.1 Algorithm Development

I developed and compared two primary architectures for this task:

1. **Baseline U-Net:** A custom-built convolutional autoencoder with skip connections to preserve spatial resolution.
2. **ResNet-UNet (Transfer Learning):** An advanced architecture replacing the standard encoder with a pre-trained **ResNet-18**. This leveraged semantic knowledge from ImageNet (e.g., knowing that "grass" is usually green) to improve color prediction accuracy.

2.2 Theoretical Background: The "Lab" Trick

Standard RGB models struggle with colorization because color and brightness are entangled. I implemented the CIE Lab color space, which separates Lightness (L) from Chrominance (a, b).

- **Input:** L channel (Lightness), normalized to $[-1, 1]$.

- **Output:** ab channels (Green-Red, Blue-Yellow), normalized to $[-1, 1]$.
- **Loss Function:** I employed **L1 Loss** (Mean Absolute Error) as the primary objective function, as it encourages sharper images compared to MSE.

3. Detailed Contribution

3.1 Data Engineering & Pipeline

I engineered a robust data pipeline capable of handling the COCO 2017 dataset (118,000 images), ensuring we met the project's GPU requirement.

- **Cloud Infrastructure:** I set up an AWS EC2 instance (T4/V100 GPU) and wrote `download_dataset.py` to fetch and extract 18GB of data directly to the cloud.
- **Custom DataLoader:** I implemented a PyTorch Dataset class that performs on-the-fly RGB-to-Lab conversion. This eliminated the need to store duplicate grayscale datasets, optimizing storage efficiency.

3.2 Curriculum Learning Strategy

To overcome the "Regression to the Mean" problem (where models predict gray/sepia colors to minimize error), I designed and implemented a systematic 5-Phase Curriculum Learning Strategy for the Baseline U-Net. This involved progressively adjusting the learning rate and loss functions over 50 epochs to guide the model from structural understanding to color vibrancy.

- **Phase 1: Initial Training (Epochs 1-5):**
 - **Objective:** Establish basic semantic segmentation (sky vs. ground, skin vs. clothing).
 - **Settings:** High Learning Rate ($2e-4$), Standard L1 Loss.
 - **Outcome:** The model successfully converged on shapes but outputted monochromatic (sepia) predictions.
- **Phase 2: Fine-Tuning (Epochs 6-15):**
 - **Objective:** Sharpen edges and textures without altering color logic.
 - **Settings:** Lower Learning Rate ($5e-5$), Standard L1 Loss.
 - **Outcome:** Significant improvement in SSIM (Structural Similarity) scores; edges became crisp, preventing color bleeding.
- **Phase 3: Refined Training (Epochs 16-30):**
 - **Objective:** Deep convergence on structural details.
 - **Settings:** Very Low Learning Rate ($2e-5$).

- **Outcome:** The model reached its peak structural performance (SSIM ~0.49), but colors remained conservative due to the L1 loss penalty.
- **Phase 4: Vibrant Training (Epochs 31-35):**
 - **Objective:** Penalize the "Safe Gray" strategy.
 - **Technique:** I implemented a **Linearly Weighted Loss**. The loss for each pixel was multiplied by a factor proportional to its ground-truth saturation (Weight = $1 + 2 \times \text{Saturation}$).
 - **Outcome:** The model began to predict distinct hues (greens and blues) instead of mud brown.
- **Phase 5: Aggressive Training (Epochs 36-50):**
 - **Objective:** Force maximum color boldness.
 - **Technique:** I implemented an **Exponential Weighted Loss** (Weight = $e^{3 \times \text{Saturation}}$). This heavily penalized the model for predicting gray when the ground truth was vibrant.
 - **Outcome:** This successfully broke the "sepia trap" for many objects, though it highlighted the need for semantic understanding (ResNet) to resolve ambiguous objects like hats or bags.

3.3 Final Architecture Upgrade (ResNet-18)

To further improve semantic consistency, I refactored the model to use Transfer Learning. I implemented ResNetUNet, utilizing a pre-trained ResNet-18 as the encoder backbone. This significantly improved convergence speed and edge detection compared to the baseline U-Net.

3.4 Deployment & Application

I built the final user-facing application using Streamlit. The app allows users to upload a grayscale image and see the colorized result in real-time. I also implemented a Saturation Booster slider to allow users to manually correct the model's desaturation bias, a common issue in regression-based colorization.

4. Results

I evaluated the models using L1 Loss (lower is better), PSNR (Peak Signal-to-Noise Ratio, higher is better), and SSIM (Structural Similarity Index, higher is better).

4.1 Quantitative Comparison

The ResNet-based model outperformed the standard U-Net in both efficiency and structural accuracy, achieving better metrics in fewer epochs.

Model Variant	Training Epochs	Validation Loss (L1)	PSNR (dB)	SSIM
Baseline U-Net	50	0.0639	27.84	0.458
ResNet-UNet	20	0.0617	28.00	0.496

4.2 Qualitative Results

To visually assess the model's performance, I compared the generated outputs against the ground truth on unseen test images from the COCO validation set.

Test Results:



Analysis: As seen in Figures, the ResNet-UNet successfully recovered complex semantic colors while handling the inherent ambiguity of the task.

- **Test Case 1 (Zebras):** The model correctly identified the zebras as black-and-white animals, avoiding the common failure case where models "bleed" green from the background onto the animal. Interestingly, the model predicted yellowish-brown grass compared to the green ground truth. This perfectly illustrates the ill-posed nature of colorization: "grass" is semantically valid as both green and brown, and the model made a plausible semantic choice even if it differed from the specific ground truth instance.
- **Test Case 2 (Clock):** The model successfully recovered the wood texture of the clock, assigning a realistic brown hue to the cabinet. However, it struggled to capture the warm, yellow lighting cast present in the ground truth, resulting in a slightly cooler, neutral wall color. This confirms that while L1 Loss is excellent for local object consistency, it can sometimes average out global atmospheric lighting effects.

5. Summary and Conclusions

In this project, I successfully developed and deployed a deep learning pipeline for automatic image colorization. By leveraging the Lab color space and Transfer Learning with ResNet-18, I achieved a model that outperforms a standard U-Net in both convergence speed (2.5x faster) and structural accuracy (SSIM 0.496).

I learned that while L1 loss functions are stable, they inherently encourage desaturation in generative tasks. Future improvements would involve shifting from a regression-based approach to a **Generative Adversarial Network (GAN)** (e.g., Pix2Pix), where a Discriminator network can specifically penalize lack of vibrancy.

6. Percentage of Code Attribution

The calculation below adheres to the formula provided in the project instructions:

$$\text{Percentage} = (\text{Copied} - \text{Modified}) / (\text{Copied} + \text{Added}) * 100$$

Code Breakdown:

- **Copied Code (~380 lines):** Standard boilerplate for U-Net architecture foundations, VGG feature extraction setup, and basic Streamlit UI components found in documentation and open-source tutorials.
- **Modified Code (~120 lines):** Significant adaptations to the standard U-Net to accept a 1-channel input (L) and output 2 channels (ab), integrating the ResNet-18 backbone, and customizing the data loader for on-the-fly Lab conversion.

- **Added Code (~400 lines):** The custom training loops, the specific "Saturation Booster" logic in the app, the high-res image fusion pipeline, and the custom weighted loss functions were written entirely from scratch.

Calculation:

- Total Lines = $380 + 400 = 780$
- Percentage Copied = $(380 - 120) / (380 + 400) * 100$
- Percentage Copied = $260 / 780 * 100 \approx \mathbf{33.3\%}$

Conclusion: Approximately **33%** of the code was derived from external sources, while **67%** represents my own original work and modifications, demonstrating a significant individual contribution to the project.

7. References

1. Zhang, R., Isola, P., & Efros, A. A. (2016). Colorful Image Colorization. *ECCV*.
2. Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*.
3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*.
4. COCO Consortium. (2017). Common Objects in Context (COCO) Dataset.